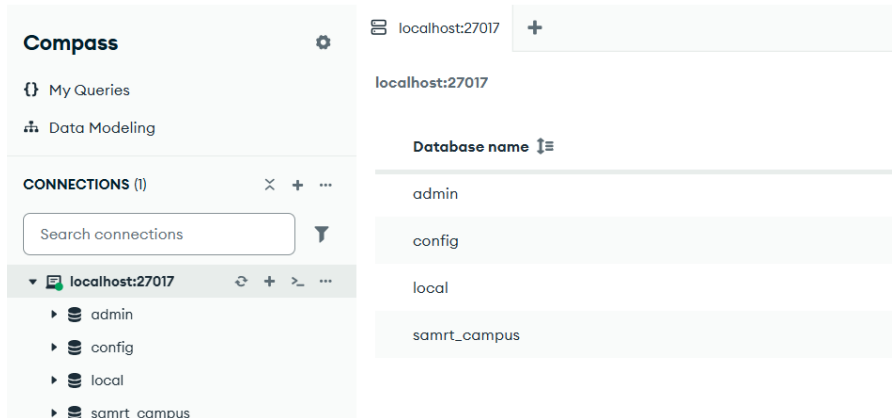
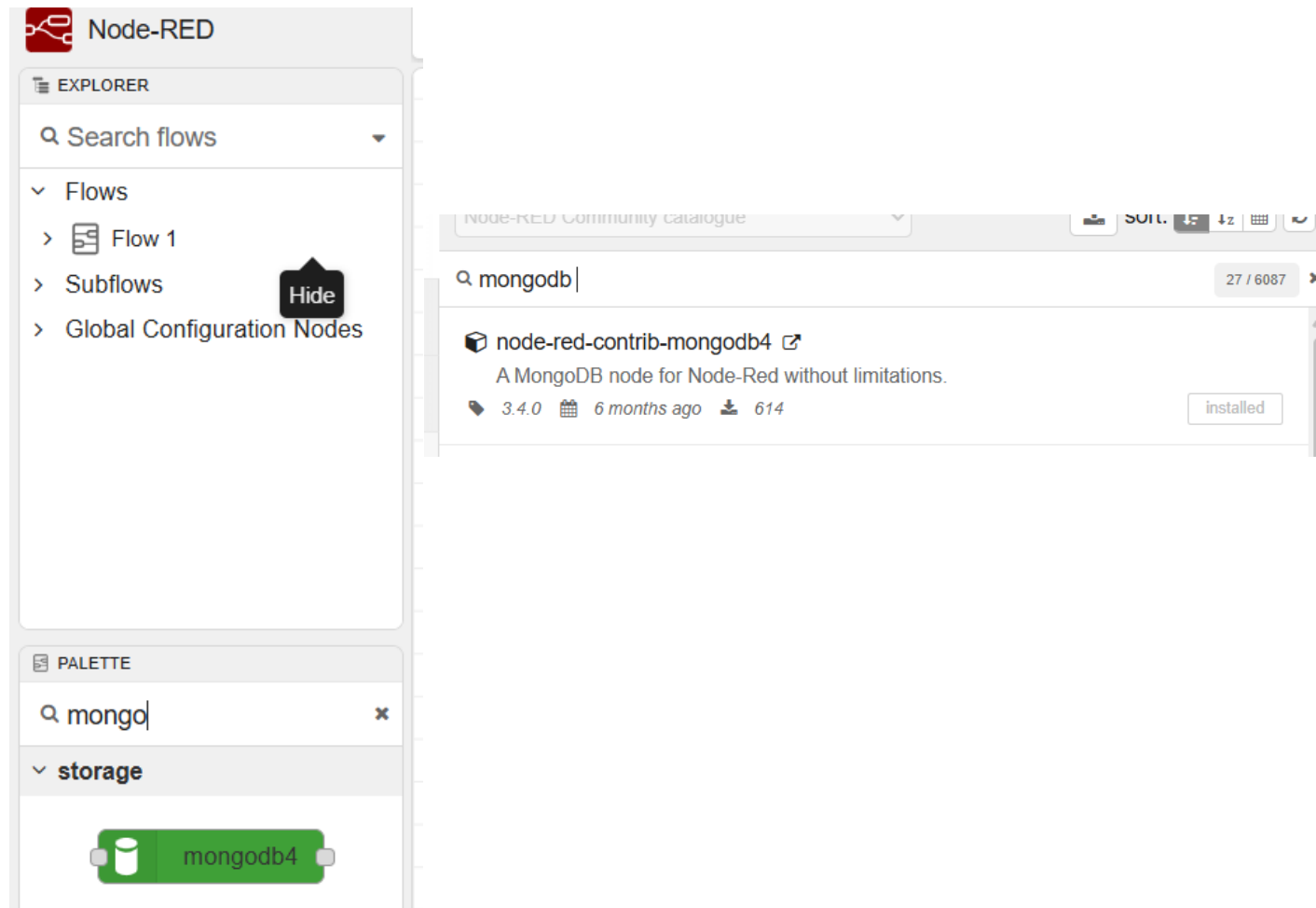


Exercise 6: Database Foundations

Step 1: Deploy MongoDB



Step 2: Install MongoDB Nodes in Node-RED



Step 3: Configure the Database Connection

Edit mongodb4 node > Edit mongodb4-client node

Delete Cancel

Properties

Name MongoDB Docker

Simple Connection URI Advanced Connection URI

URI mongodb://host.docker.internal:27017/smart_campus?directCor

Database smart_campus

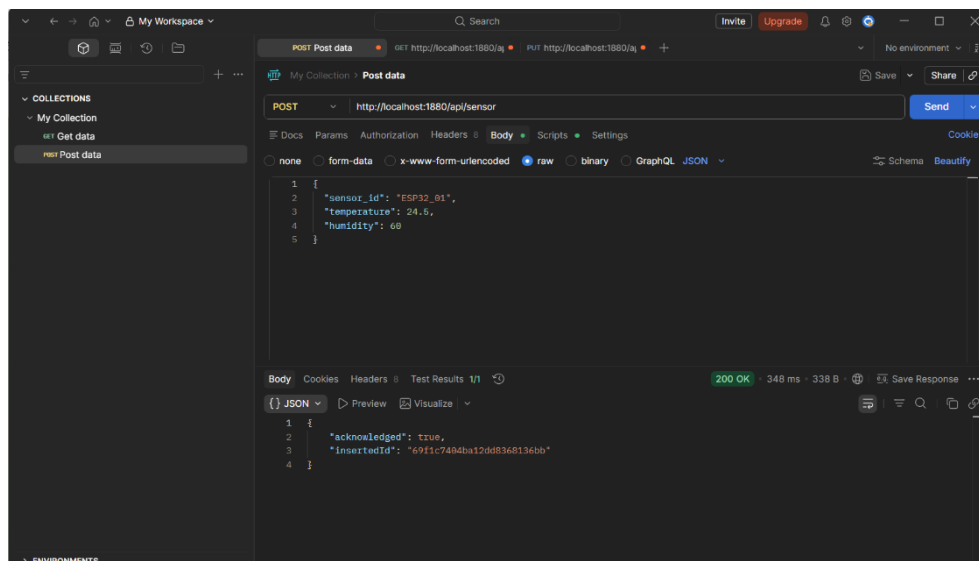
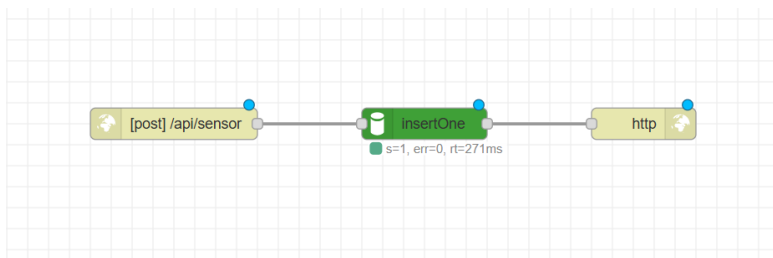
AppName optional

AuthMech DEFAULT

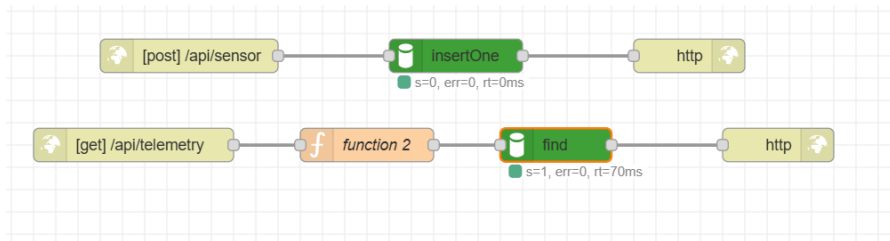
AuthSource optional

Step 4: Enhancing REST APIs to support full CRUD functionality.

Part A: CREATE (HTTP POST)



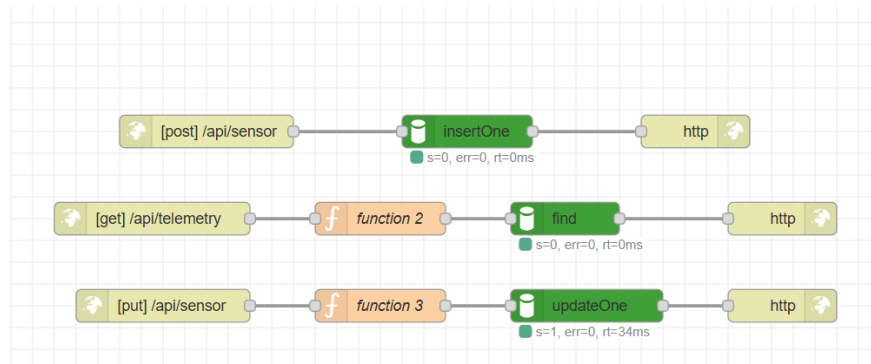
Part B: READ (HTTP GET)



The screenshot shows a web browser window with the address bar displaying `http://localhost:1880/api/telemetry`. The browser is set to GET. The response is a 200 OK status, received in 94 ms. The response body is displayed in JSON format, showing an array of four telemetry records. Each record contains an `_id`, `sensor_id`, `temperature`, and `humidity`.

```
1 [
2   {
3     "_id": "69ef7936ab5fdf40f3e59c47",
4     "sensor_id": "ESP32_01",
5     "temperature": 24.5,
6     "humidity": 60
7   },
8   {
9     "_id": "69ef797eab5fdf40f3e59c48",
10    "sensor_id": "ESP32_01",
11    "temperature": 24.5,
12    "humidity": 60
13  },
14  {
15    "_id": "69f07444e0f39dca6a8e2f2d",
16    "sensor_id": "ESP32_01",
17    "temperature": 24.5,
18    "humidity": 60
19  },
20  {
21    "_id": "69f076f2e0f39dca6a8e2f2e",
22    "sensor_id": "ESP32_01",
23    "temperature": 24.5,
24    "humidity": 60
25  }
26 ]
```

Part C: UPDATE & DELETE (Advanced Logic)



POST Post data • GET http://localhost:1880/api/telemetry • PUT http://localhost:1880/api/sensor

PUT http://localhost:1880/api/sensor

Docs Params Authorization Headers 7 Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers 8 Test Results 200 OK • 54 ms

JSON Preview Visualize

```
1 {
2   "acknowledged": true,
3   "modifiedCount": 0,
4   "upsertedId": null,
5   "upsertedCount": 0,
6   "matchedCount": 0
7 }
```

Question 1: Dynamic Schema Generation

-No Fixed Rules: MongoDB is "schema-less," meaning you don't have to set up a rigid, permanent table structure before saving data.

-Mix and Match Data: You can save one sensor reading with just temperature and humidity, and the very next reading can include extra fields like pressure or battery power without breaking anything.

-Perfect for Prototyping: When building early IoT projects, sensor designs change constantly. MongoDB lets you update your data formats instantly on the fly without wasting time redesigning a strict database structure.

Question 2: Middleware Abstraction

-Node-RED acts as a Guard: Instead of letting sensors or tools like Postman talk directly to your database, they talk to Node-RED. Node-RED checks, cleans, and controls the data before passing it along.

-Hiding the Database: Leaving MongoDB's default connection port (27017) open to the public internet makes it easy targets for hackers looking to steal, delete, or attack your data.

-Smaller Target: Keeping the database private and putting Node-RED in front of it blocks direct outside access, giving you a single, secure gateway to manage and log everything.